BABU BANARSI DAS UNIVERSITY LUCKNOW U.P.



Lab Manual

on

Deep Learning Programming Lab**(NCCML4751)**

For

B.Tech2nd Year**(Sem.** 3rd**)**

Session(2024-25)

BBDU

BabuBanarasi Das University

Subject:Deep learningLab(NCCML4751) Program:B.Tech.CCML2nd Year(**Sem-**3rd)

INDEX

Sr. No.	Title	Page No.
1.	Write a program to implement k-Nearest Neighbour algorithm to classify the iris data set. Print both correct and wrong predictions	4-5
2.	Build a Logistic Regression model that answers the question: "what sorts of people were more likely to survive?" using passenger data (ie name, age, gender, socio-economic class, etc). in Titanic datase	6-9
3.	Build a model to digit recognition of MNIST dataset using Support Vector Machine. Also print the confusion matrix	10-11
4.	Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.	12-13
5.	Construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease Data Set	14-16
6.	Write a program to classify retinal damage from OCT Scan dataset using a pre- trained VGG16 Model	17-19
7.	Write a program to visualization of each species of iris dataset using Liner Regression Model	20-24
8.	Build a CNN Model to identify Image from the CIFAR-10 Dataset. Calculate the accuracy, precision, and recall for your data set	25-26
9.	Construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease Data Set.	27-28
10.	Consider the "airline-passengers.csv "dataset. Write a program to implement LSTM (Short Long-term Memory) Network, the task is to predict the number of international airline passengers in units of 1,000. The data ranges from January 1949 to December 1960, or 12 years, with 144 observations	29-30
11	Project Statement Project Title – Stock Market Prediction Stock price prediction is one among the complex machine learning problems. It depends on a large number of factors which contribute to changes in the supply CO3,4 and demand. This paper presents the technical analysis of the various strategies proposed in the past, for predicting the price of a stock, and evaluation of a novel approach for the same. Stock prices are represented as time series data and neural networks are trained to learn the patterns from trends. Along with	31-32

	the numerical analysis of the stock trend, this research also considers the textual analysis of it by analyzing the public sentiment from online news sources and blogs. Utilizing both this information, a merged hybrid model is built which can predict the stock trend more accurately	

BBDU

ProgramNo.:1

Write a program to implement k-Nearest Neighbour algorithm to classify the iris data set. Print both correct and wrong predictions

Python Program to Implement and Demonstrate KNN Algorithm

```
importnumpy as np
import pandas as pd
fromsklearn.neighbors import KNeighborsClassifier
fromsklearn.model_selection import train_test_split
fromsklearn import metrics

names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'Class']

# Read dataset to pandas dataframe
dataset = pd.read_csv("9-dataset.csv", names=names)
X = dataset.iloc[:, :-1]
y = dataset.iloc[:, -1]
print(X.head())
Xtrain, Xtest, ytrain, ytest = train_test_split(X, y, test_size=0.10)

classifier = KNeighborsClassifier(n_neighbors=5).fit(Xtrain, ytrain)

ypred = classifier.predict(Xtest)
i = 0
print ("\n-------------------------------------------------------------------------")
print ('%-25s %-25s %-25s' % ('Original Label', 'Predicted Label', 'Correct/Wrong'))
print ("-------------------------------------------------------------------------")
for label in ytest:
print ('%-25s %-25s' % (label, ypred[i]), end="")
if (label == ypred[i]):
print (' %-25s' % ('Correct'))
else:
print (' %-25s' % ('Wrong'))
   i = i + 1
print ("-------------------------------------------------------------------------")
print("\nConfusion Matrix:\n",metrics.confusion_matrix(ytest, ypred))
print ("-------------------------------------------------------------------------")
print("\nClassification Report:\n",metrics.classification_report(ytest, ypred))
print ("-------------------------------------------------------------------------")
print('Accuracy of the classifer is %0.2f' % metrics.accuracy_score(ytest,ypred))
print ("-------------------------------------------------------------------------")
```

Output

```
   sepal-length  sepal-width  petal-length  petal-width
0          5.1          3.5           1.4          0.2
1          4.9          3.0           1.4          0.2
2          4.7          3.2           1.3          0.2
3          4.6          3.1           1.5          0.2
4          5.0          3.6           1.4          0.2
-------------------------------------------------------------------------
Original Label           Predicted Label          Correct/Wrong
```

```
-Iris-versicolor          Iris-versicolor          Correct
Iris-virginica          Iris-versicolor          Wrong
Iris-virginica          Iris-virginica          Correct
Iris-versicolor          Iris-versicolor          Correct
Iris-setosa          Iris-setosa          Correct
Iris-versicolor          Iris-versicolor          Correct
Iris-setosa          Iris-setosa          Correct
Iris-setosa          Iris-setosa          Correct
Iris-virginica          Iris-virginica          Correct
Iris-virginica          Iris-versicolor          Wrong
Iris-virginica          Iris-virginica          Correct
Iris-setosa          Iris-setosa          Correct
Iris-virginica          Iris-virginica          Correct
Iris-virginica          Iris-virginica          Correct
Iris-versicolor          Iris-versicolor          Correct
----------------------------------------------------------------------
Confusion Matrix:
 [[4 0 0]
 [0 4 0]
 [0 2 5]]
----------------------------------------------------------------------Classification Report:
precision    recall  f1-score   support

    Iris-setosa       1.00      1.00      1.00         4
Iris-versicolor       0.67      1.00      0.80         4
 Iris-virginica       1.00      0.71      0.83         7

avg / total       0.91      0.87      0.87        15
```

ProgramNo.:2

Build a Logistic Regression model that answers the question: "what sorts of people were more likely to survive?" using passenger data (ie name, age, gender, socio-economic class, etc). in Titanic datase

```
# This Python 3 environment comes with many helpful analytics libraries installed
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

importnumpyasnp# linear algebra
importpandasaspd# data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will list all files under the input directory

importos
fordirname,_,filenamesinos.walk('/kaggle/input'):
forfilenameinfilenames:
print(os.path.join(dirname,filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that gets preserved as output when you create a version using "Save & Run All"
# You can also write temporary files to /kaggle/temp/, but they won't be saved outside of the current session
/kaggle/input/titanic/train.csv
/kaggle/input/titanic/test.csv
/kaggle/input/titanic/gender_submission.csv

#libraries for visualisation
importmatplotlib.pyplotasplt
importseabornassns
importcufflinksascf
importplotly.expressaspx
%matplotlib inline

# Library for feature Scaling
fromsklearn.preprocessingimportRobustScaler

#Libraries for logistic Regression Model Building
fromsklearn.linear_modelimportLogisticRegression
fromsklearn.model_selectionimporttrain_test_split


#Library for evaluation
fromsklearnimportmetrics
fromsklearn.metricsimportconfusion_matrix
fromsklearn.metricsimportclassification_report
```

. Importing Training and Test Dataset¶
As per Titanic - Machine Learning from Disaster dataset there two seperatecsv files for training and test data.

training data set is imported to train_data and test data is imported to test_data

In [3]:
linkcode
```
train_data=pd.read_csv('../input/titanic/train.csv')
test_data=pd.read_csv('../input/titanic/test.csv')
```

```
Index(['PassengerId', 'Survived', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp',
       'Parch', 'Ticket', 'Fare', 'Cabin', 'Embarked'],
dtype='object')
```

In [5]:
```
train_data.head()
```

Out[5]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

In [6]:
```
test_data.columns
```

Out[6]:
```
Index(['PassengerId', 'Pclass', 'Name', 'Sex', 'Age', 'SibSp', 'Parch',
       'Ticket', 'Fare', 'Cabin', 'Embarked'],
dtype='object')
```

In [7]:
linkcode
test_data.head()
Out[7]:

	PassengerId	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	892	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292	NaN	Q
1	893	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000	NaN	S
2	894	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875	NaN	Q
3	895	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625	NaN	S
4	896	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298	12.2875	NaN	S

Dataset details are below :

- PassengerId - Indicates the passenger number
- Survived - Indicates if the passenger travelled is survived or not.
- Pclass - Indicates which class passenger belows
- Name - Provides Names details of passenger
- Sex - Indicates sex/gender of passenger
- Age - Indicates age of passenger
- SibSp - Provides information about number of sibiling/spouse accompaning
- Parch - Provides information about number of parents / children aboard the Titanic
- Ticket - Ticket number
- Fare - Passenger fare
- Cabin - Provides information about Cabin number, NaN specifies no cabin allocated
- Embarked - Port of Embarkation

PROGRAM-3

Build a model to digit recognition of MNIST dataset using Support Vector Machine. Also print the confusion matrix

```
importnumpyasnp
importpandasaspd
importmatplotlib.pyplotasplt
importseabornassns
fromsklearnimportlinear_model
fromsklearn.model_selectionimporttrain_test_split
importgc
importcv2
```

In [2]:

```
linkcode
# read the dataset
digits=pd.read_csv("../input/mnist-svm-m4/train.csv")
digits.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 42000 entries, 0 to 41999
Columns: 785 entries, label to pixel783
dtypes: int64(785)
memory usage: 251.5 MB

digits.head()
```

Out[3]:

	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	...	pixel774	pixel775	pixel776	pixel777	pixel778	pixel779	pixel780	pixel781	pixel782	pixel783
0	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
2	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
3	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0
4	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0	0	0	0	0	0	0

	label	pixel0	pixel1	pixel2	pixel3	pixel4	pixel5	pixel6	pixel7	pixel8	...	pixel774	pixel775	pixel776	pixel777	pixel778	pixel779	pixel780	pixel781	pixel782	pixel783
											.										

5 rows × 785 columns

In [4]:

```
four=digits.iloc[3,1:]
four.shape
```

(784,)

atplotlib.image.AxesImage at 0x7fae1b355080>

```
confusion=metrics.confusion_matrix(y_true=y_test,y_pred=predictions)

# measure accuracy
test_accuracy=metrics.accuracy_score(y_true=y_test,y_pred=predictions)

print(test_accuracy,"\n")
print(confusion)
```

0.924973544973545

```
[[3587    0   10   10    5   15   50   12   25    1]
 [   0 4108   14   16    5    3    6   18   10    5]
 [  24   23 3407   65   44    5   36  123   54    9]
 [   4   21   86 3502    5   89   11   73   76   33]
 [   3   11   36    7 3450   13   23   43    6  110]
 [  20   29   14  114   18 3020   79   53   36   35]
 [  31   12   11    1   14   34 3521   44   25    0]
 [   4   28   27    8   36    7    1 3739    7   97]
 [  14   59   32   80   22   97   25   44 3251   41]
 [  23   13   13   50   98    7    0  176   19 3379]]
```

PROGRAM-4

Write a program to implement the naïve Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.

```
# import necessary libarities
import pandas as pd
from sklearn import tree
from sklearn.preprocessing import LabelEncoder
from sklearn.naive_bayes import GaussianNB

# load data from CSV
data=pd.read_csv('tennisdata.csv')
print("THe first 5 values of data is :\n",data.head())
```

```
THe first 5 values of data is :
    Outlook Temperature Humidity  WindyPlayTennis
0     Sunny         Hot     High  False         No
1     Sunny         Hot     High   True         No
2  Overcast         Hot     High  False        Yes
3     Rainy        Mild     High  False        Yes
4     Rainy        Cool   Normal  False        Yes
```

```
In [2]:
# obtain Train data and Train output
X =data.iloc[:,:-1]
print("\nThe First 5 values of train data is\n",X.head())
```

```
The First 5 values of train data is
    Outlook Temperature Humidity  Windy
0     Sunny         Hot     High  False
1     Sunny         Hot     High   True
2  Overcast         Hot     High  False
3     Rainy        Mild     High  False
4     Rainy        Cool   Normal  False
```

```
In [3]:
y =data.iloc[:,-1]
print("\nThe first 5 values of Train output is\n",y.head())
```

```
The first 5 values of Train output is
 0     No
1     No
2    Yes
3    Yes
4    Yes
Name: PlayTennis, dtype: object
```

```
In [4]:
# Convert then in numbers
le_outlook=LabelEncoder()
X.Outlook=le_outlook.fit_transform(X.Outlook)

le_Temperature=LabelEncoder()
```

```
X.Temperature=le_Temperature.fit_transform(X.Temperature)

le_Humidity=LabelEncoder()
X.Humidity=le_Humidity.fit_transform(X.Humidity)

le_Windy=LabelEncoder()
X.Windy=le_Windy.fit_transform(X.Windy)

print("\nNow the Train data is :\n",X.head())
```

Now the Train data is :

	Outlook	Temperature	Humidity	Windy
0	2	1	0	0
1	2	1	0	1
2	0	1	0	0
3	1	2	0	0
4	1	0	1	0

In [5]:

```
le_PlayTennis=LabelEncoder()
y =le_PlayTennis.fit_transform(y)
print("\nNow the Train output is\n",y)
```

Now the Train output is

```
 [0 0 1 1 1 0 1 0 1 1 1 1 1 0]
```

In [6]:

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test=train_test_split(X,y, test_size=0.20)

classifier=GaussianNB()
classifier.fit(X_train,y_train)

from sklearn.metrics import accuracy_score
print("Accuracy is:",accuracy_score(classifier.predict(X_test),y_test))
```

Accuracy is: 0.6666666666666666

PROGRAM-5

Construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease Data Set

A Bayesian network is a directed acyclic graph in which each edge corresponds to a conditional dependency, and each node corresponds to a unique random variable. Bayesian network consists of two major parts: a directed acyclic graph and a set of conditional probability distributions

1 The directed acyclic graph is a set of random variables represented by nodes.

2 The conditional probability distribution of a node (random variable) is defined for every possible outcome of the preceding causal node(s).For illustration, consider the following example. Suppose we attempt to turn on our computer, but the computer does not start (observation/evidence). We would like to know which of the possible causes of computer failure is more likely. In this simplified illustration, we assume only two possible causes of this misfortune: electricity failure and computer malfunction.

The corresponding directed acyclic graph is depicted in below figure. The goal is to calculate the posterior conditional probability distribution of each of the possible unobserved causes given the observed evidence, i.e. P [Cause | Evidence].

Data Set:

Title: Heart Disease Databases

The Cleveland database contains 76 attributes, but all published experiments refer to using a subset of 14 of them. In particular, the Cleveland database is the only one that has been used by ML researchers to this date. The "Heartdisease" field refers to the presence of heart diseasein the patient. It is integer valued from 0 (no presence) to 4.

Database: 0 1 2 3 4 Total

Cleveland: 164 55 36 35 13 303

Attribute Information:

1. age: age in years

2. sex: sex (1 = male; 0 = female)

3. cp: chest pain type

Value 1: typical angina

Value 2: atypical angina

Value 3: non-anginal pain

Value 4: asymptomatic

4. trestbps: resting blood pressure (in mm Hg on admission to the hospital)

5. chol: serum cholestoral in mg/dl

6. fbs: (fasting blood sugar > 120 mg/dl) (1 = true; 0 = false)

7. restecg: resting electrocardiographic results

Value 0: normal

Value 1: having ST-T wave abnormality (T wave inversions and/or ST elevation or depression of > 0.05 mV)

Value 2: showing probable or definite left ventricular hypertrophy by Estes' criteria

8. thalach: maximum heart rate achieved

9. exang: exercise induced angina (1 = yes; 0 = no)
10. oldpeak = ST depression induced by exercise relative to rest
11.slope: the slope of the peak exercise ST segment
Value 1: upsloping
Value 2: flat
Value 3: downsloping
12. ca = number of major vessels (0-3) colored by flourosopy
13. thal: 3 = normal; 6 = fixed defect; 7 = reversable defect
14.Heartdisease: It is integer valued from 0 (no presence) to 4. Diagnosis of heart disease (angiographic disease status)
Some instance from the dataset:
age sex cptrestbpscholfbsrestecgthalachexangoldpeak slope cathalHeartdisease
63 1 1 145 233 1 2 150 0 2.3 3 0 6 0
67 1 4 160 286 0 2 108 1 1.5 2 3 3 2
67 1 4 120 229 0 2 129 1 2.6 2 2 7 1
41 0 2 130 204 0 2 172 0 1.4 1 0 3 0
62 0 4 140 268 0 2 160 0 3.6 3 2 3 3
60 1 4 130 206 0 2 132 1 2.4 2 2 7 4
Program:

```
importnumpy as np
importcsv
import pandas as pd
frompgmpy.models import BayesianModel
frompgmpy.estimators import MaximumLikelihoodEstimator
frompgmpy.inference import VariableElimination
#read Cleveland Heart Disease data
heartDisease = pd.read_csv('heart.csv')
heartDisease = heartDisease.replace('?',np.nan)
#display the data
print('Few examples from the dataset are given below')
print(heartDisease.head())
#Model Bayesian Network
Model=BayesianModel([('age','trestbps'),('age','fbs'),
('sex','trestbps'),('exang','trestbps'),('trestbps','heartdise
ase'),('fbs','heartdisease'),('heartdisease','restecg'),
('heartdisease','thalach'),('heartdisease','chol')])
#Learning CPDs using Maximum Likelihood Estimators
print('\n Learning CPD using Maximum likelihood estimators')
model.fit(heartDisease,estimator=MaximumLikelihoodEstimator)
# Inferencing with Bayesian Network
```

BBDU

```
print('\n Inferencing with Bayesian Network:')
HeartDisease_infer = VariableElimination(model)
#computing the Probability of HeartDisease given Age
print('\n 1. Probability of HeartDisease given Age=30')
q=HeartDisease_infer.query(variables=['heartdisease'],evidence
={'age':28})
print(q['heartdisease'])
#computing the Probability of HeartDisease given cholesterol
print('\n 2. Probability of HeartDisease given cholesterol=100')
q=HeartDisease_infer.query(variables=['heartdisease'],evidence
={'chol':100})
print(q['heartdisease']
```

BBDU

PROGRAM:6

Write a program to classify retinal damage from OCT Scan dataset using a pre- trained VGG16Model

About Retinal OCT

Retinal optical coherence tomography (OCT) is an imaging technique used to capture high-resolution cross sections of the retinas of living patients. Approximately 30 million OCT scans are performed each year, and the analysis and interpretation of these images takes up a significant amount of time (Swanson and Fujimoto, 2017

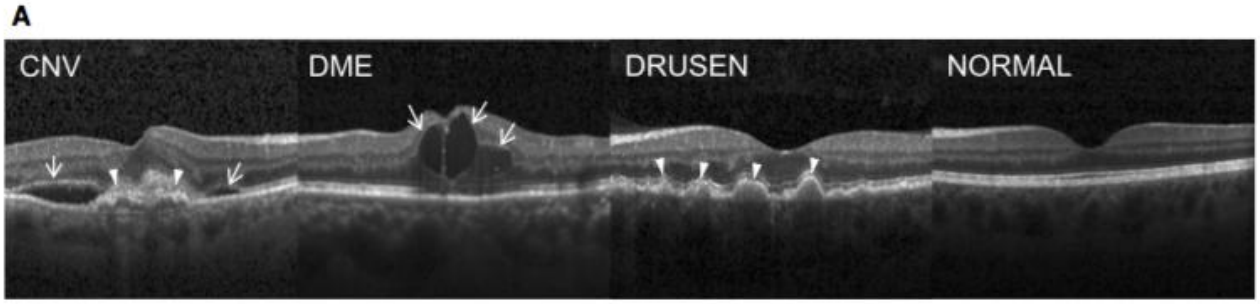


Figure 2. Representative Optical Coherence Tomography Images and the Workflow Diagram [Kermany et. al. 2018]

(A) (Far left) choroidal neovascularization (CNV) with neovascular membrane (white arrowheads) and associated subretinal fluid (arrows). (Middle left) Diabetic macular edema (DME) with retinal-thickening-associated intraretinal fluid (arrows). (Middle right) Multiple drusen (arrowheads) present in early AMD. (Far right) Normal retina with preserved foveal contour and absence of any retinal fluid/edema

```
from__future__importprint_function,division
importtorch
importtorch.nnasnn
importtorch.optimasoptim
fromtorch.optimimportlr_scheduler
fromtorch.autogradimportVariable
importnumpyasnp
importtorchvision
fromtorchvisionimportdatasets,models,transforms
importmatplotlib.pyplotasplt
importtime
importos
importcopy

plt.ion()

use_gpu=torch.cuda.is_available()
ifuse_gpu:
print("Using CUDA")
```

Dataset loader¶

The dataset is divided in three categories: training, validation and test.

The first one will be, obviously, used for trainig; the validation set will be used to measure the model performance during training and the test set will be used to evaluate our model performance once the training has finished.

Note: These three sets should all contain different images.
Loading this dataset with pytorch is really easy using ImageFolder as the labels are specified by the folders names.

```
data_dir='../input/d/obulisainaren/retinal-oct-c8/RetinalOCT_Dataset'

TRAIN='train'
VAL='val'
TEST='test'

# VGG-16 Takes 224x224 images as input, so we resize all of them
data_transforms={
TRAIN:transforms.Compose([
# Data augmentation is a good practice for the train set
# Here, we randomly crop the image to 224x224 and
# randomly flip it horizontally.
transforms.RandomResizedCrop(224),
transforms.RandomHorizontalFlip(),
transforms.ToTensor(),
]),
VAL:transforms.Compose([
transforms.Resize(256),
transforms.CenterCrop(224),
transforms.ToTensor(),
]),
TEST:transforms.Compose([
transforms.Resize(256),
transforms.CenterCrop(224),
transforms.ToTensor(),
])
}

image_datasets={
x:datasets.ImageFolder(
os.path.join(data_dir,x),
transform=data_transforms[x]
)
forxinin[TRAIN,VAL,TEST]
}

dataloaders={
x:torch.utils.data.DataLoader(
image_datasets[x],batch_size=8,
```

BBDU

```
shuffle=True,num_workers=4
)
forxinTRAIN,VAL,TEST]
}

dataset_sizes={x:len(image_datasets[x])forxinTRAIN,VAL,TEST]}

forxinTRAIN,VAL,TEST]:
print("Loaded {} images under {}".format(dataset_sizes[x],x))

print("Classes: ")
class_names=image_datasets[TRAIN].classes
print(image_datasets[TRAIN].classes)
Loaded 18400 images under train
Loaded 2800 images under val
Loaded 2800 images under test
Classes:
['AMD', 'CNV', 'CSR', 'DME', 'DR', 'DRUSEN', 'MH', 'NORMAL']
defimshow(inp,title=None):
inp=inp.numpy().transpose((1,2,0))
# plt.figure(figsize=(10, 10))
plt.axis('off')
plt.imshow(inp)
iftitleisnotNone:
plt.title(title)
plt.pause(0.001)

defshow_databatch(inputs,classes):
out=torchvision.utils.make_grid(inputs)
imshow(out,title=[class_names[x]forxinclasses])

# Get a batch of training data
inputs,classes=next(iter(dataloaders[TRAIN]))
show_databatch(inputs,classes)
```

BBDU

PROGRAM:07

Write a program to visualization of each species of iris dataset using Liner Regression Model

Before we begin…

Firstly, you will need to have Python installed and an Integrated Development Environment (IDE) of your choice (this is completely up to you, you may or may not want to use an IDE). I am using Jupyter Notebook as it is by far the best IDE for data visualisation/manipulation and Machine Learning in my opinion.

Secondly, you will also need to install **pandas**, **NumPy**, **scikit-learn (sklearn)**, **matplotlib** and finally **seaborn**.

Run pip/pip3/conda install on your command line to install these packages as such.

```
pip/pip3/conda install pandas
pip/pip3/conda install numpy
pip/pip3/conda install scikit-learn
pip/pip3/conda install sklearn
pip/pip3/conda install matplotlib
pip/pip3/conda install seaborn
```

Run the command according to the version of your pip installer and/or if you are using the anaconda package management system.Finally, to check if you have successfully downloaded the librarie, you can either type pip/pip3 freeze **in the command line**
or check the Anaconda Navigator Environments.

The Iris Dataset

There are 3 species in the Iris genus namely Iris Setosa, Iris Versicolor and Iris Virginica and 50 rows of data for each species of Iris flower. The column names represent the feature of the flower that was studied and recorded.

This is how I have prepared the Iris Dataset which I have loaded from sklearn.datasets. Alternatively, you could download the dataset from UCI Machine Learning Repository in the form of a CSV File.

```
# Import Dataset from sklearn
from sklearn.datasets import load_iris# Load Iris Data
iris = load_iris()# Creating pdDataFrames
iris_df = pd.DataFrame(data= iris.data, columns= iris.feature_names)
target_df = pd.DataFrame(data= iris.target, columns= ['species'])def converter(specie):
    if specie == 0:
return 'setosa'
elif specie == 1:
```

```
    return 'versicolor'
  else:
    return 'virginica'target_df['species'] = target_df['species'].apply(converter)# Concatenate the DataFrames
iris_df = pd.concat([iris_df, target_df], axis= 1)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	virginica
146	6.3	2.5	5.0	1.9	virginica
147	6.5	3.0	5.2	2.0	virginica
148	6.2	3.4	5.4	2.3	virginica
149	5.9	3.0	5.1	1.8	virginica

150 rows × 5 columns

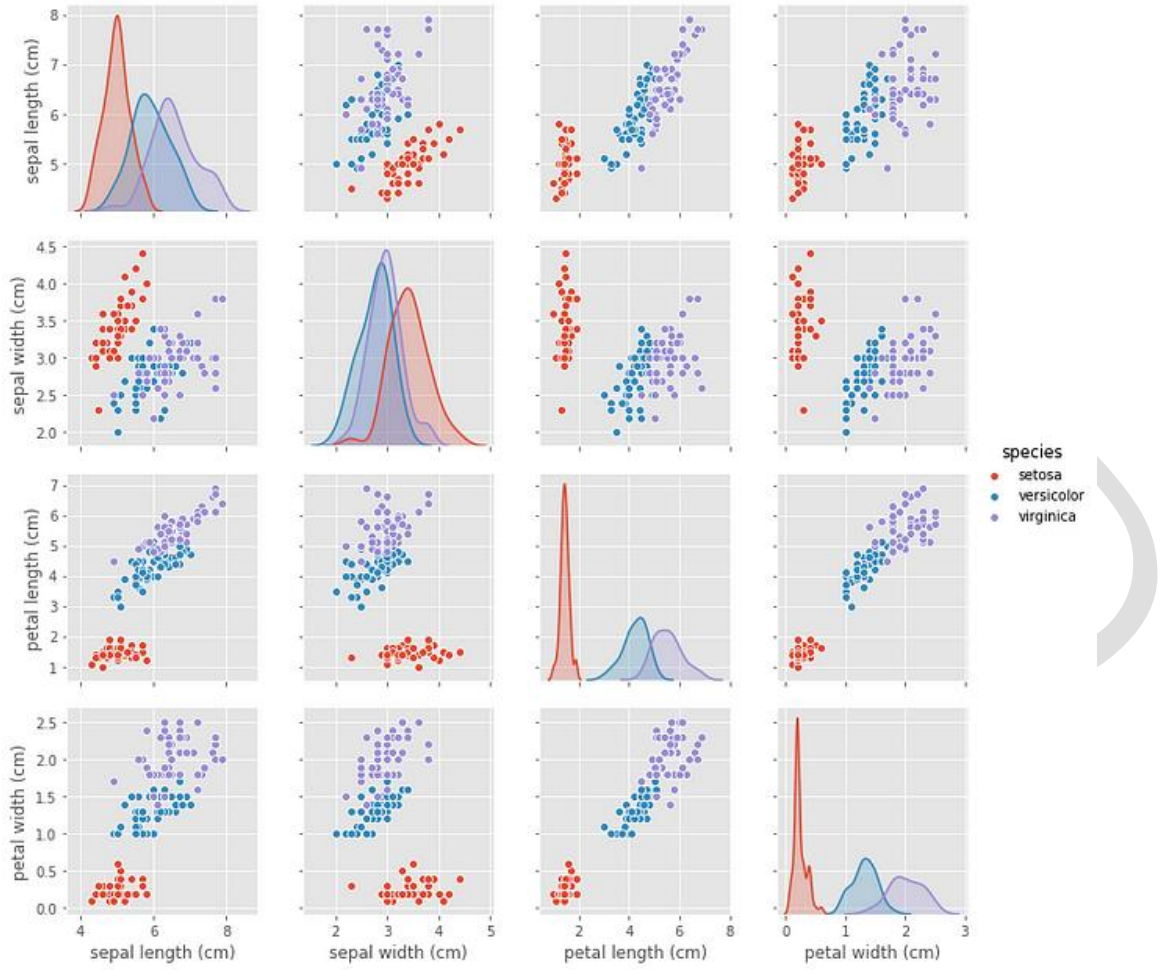
An overview of the dataset:

iris_df.describe().describe() generates descriptive statistics that summarize the central tendency, dispersion and shape of a dataset's distribution, excluding NaN values.

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

iris_df.info().info() prints a concise summary of a DataFrame.

```
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
sepal length (cm)     150 non-null float64
sepal width (cm)      150 non-null float64
petal length (cm)     150 non-null float64
petal width (cm)      150 non-null float64
species               150 non-null object
dtypes: float64(4), object(1)
```



Problem Statement: Predict the sepal length (cm) of the iris flowers

Here comes the coding part!

```
# Converting Objects to Numerical dtype
iris_df.drop('species', axis= 1, inplace= True)
target_df = pd.DataFrame(columns= ['species'], data= iris.target)
iris_df = pd.concat([iris_df, target_df], axis= 1)# Variables
X= iris_df.drop(labels= 'sepal length (cm)', axis= 1)
y= iris_df['sepal length (cm)']

# Splitting the Dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size= 0.33, random_state= 101)

# Instantiating LinearRegression() Model
lr = LinearRegression()

# Training/Fitting the Model
```

```
lr.fit(X_train, y_train)

# Making Predictions
lr.predict(X_test)
pred = lr.predict(X_test)

# Evaluating Model's Performance
print('Mean Absolute Error:', mean_absolute_error(y_test, pred))
print('Mean Squared Error:', mean_squared_error(y_test, pred))
print('Mean Root Squared Error:', np.sqrt(mean_squared_error(y_test, pred)))
```

Results of fitting the model!

```
Mean Absolute Error: 0.26498350887555133
Mean Squared Error: 0.10652500975036944
Mean Root Squared Error: 0.3263816933444176
```

Now to test…

```
iris_df.loc[6]
```

```
sepal length (cm)    4.6
sepal width (cm)     3.4
petal length (cm)    1.4
petal width (cm)     0.3
species              0.0
```

```
d = {'sepal length (cm)' : [4.6],
   'sepal width (cm)' : [3.4],
   'petal length (cm)' : [1.4],
   'petal width (cm)' : [0.3],
   'species' : 0}
test_df = pd.DataFrame(data= d)
test_df
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	4.6	3.4	1.4	0.3	0

```
pred = lr.predict(X_test)print('Predicted Sepal Length (cm):', pred[0])
print('Actual Sepal Length (cm):', 4.6)
```

```
Predicted Sepal Length (cm): 4.88749921150266
Actual Sepal Length (cm): 4.6
```

PROGRAM:8

Build a CNN Model to identify Image from the CIFAR-10 Dataset. Calculate the accuracy,precision, and recall for your data set

CIFAR is an acronym that stands for the Canadian Institute For Advanced Research and the CIFAR-10 dataset was developed along with the CIFAR-100 dataset by researchers at the CIFAR institute.

The dataset is comprised of 60,000 32×32 pixel color photographs of objects from 10 classes, such as frogs, birds, cats, ships, etc. The class labels and their standard associated integer values are listed below.

- 0: airplane
- 1: automobile
- 2: bird
- 3: cat
- 4: deer
- 5: dog
- 6: frog
- 7: horse
- 8: ship
- 9: truck

These are very small images, much smaller than a typical photograph, and the dataset was intended for computer vision research.

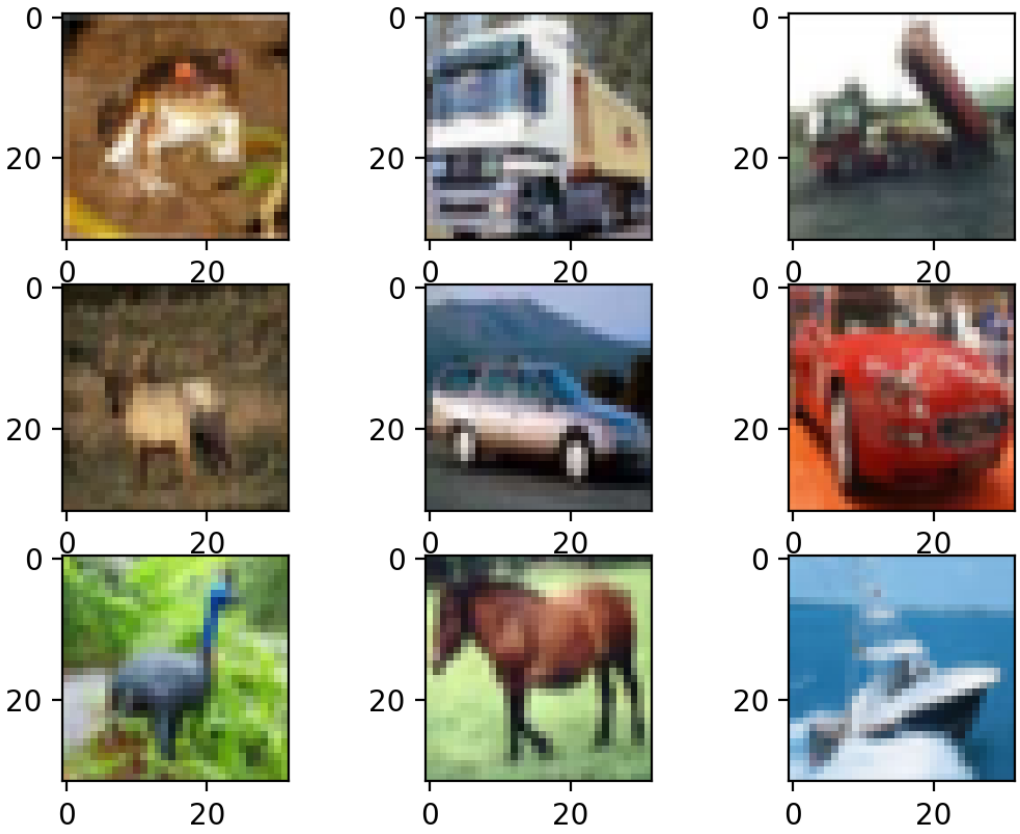
CIFAR-10 is a well-understood dataset and widely used for benchmarking computer vision algorithms in the field of machine learning. The problem is "*solved*." It is relatively straightforward to achieve 80% classification accuracy. Top performance on the problem is achieved by deep learning convolutional neural networks with a classification accuracy above 90% on the test dataset.

The example below loads the CIFAR-10 dataset using the Keras API and creates a plot of the first nine images in the training dataset.

```
# example of loading the cifar10 dataset
frommatplotlib import pyplot
fromkeras.datasets import cifar10
# load dataset
(trainX, trainy), (testX, testy) = cifar10.load_data()
# summarize loaded dataset
print('Train: X=%s, y=%s' % (trainX.shape, trainy.shape))
print('Test: X=%s, y=%s' % (testX.shape, testy.shape))
# plot first few images
for i in range(9):
 # define subplot
pyplot.subplot(330 + 1 + i)
 # plot raw pixel data
pyplot.imshow(trainX[i])
# show the figure
```

Running the example loads the CIFAR-10 train and test dataset and prints their shape.We can see that there are 50,000 examples in the training dataset and 10,000 in the test dataset and that images are indeed square with 32×32 pixels and color, with three channels.A plot of the first nine images in the dataset is also created. It is clear that the images are indeed very small compared to modern photographs; it can be challenging to see what exactly is represented in some of the images given the extremely low resolution.

This low resolution is likely the cause of the limited performance that top-of-the-line algorithms are able to achieve on the dataset.



Program-9

Construct a Bayesian network considering medical data. Use this model to demonstrate the diagnosis of heart patients using standard Heart Disease Data Set

```
importnumpy as np
import pandas as pd
importcsv
frompgmpy.estimators import MaximumLikelihoodEstimator
frompgmpy.models import BayesianModel
frompgmpy.inference import VariableElimination

heartDisease = pd.read_csv('heart.csv')
heartDisease = heartDisease.replace('?',np.nan)

print('Sample instances from the dataset are given below')
print(heartDisease.head())

print('\n Attributes and datatypes')
print(heartDisease.dtypes)

model=
BayesianModel([('age','heartdisease'),('sex','heartdisease'),('exang','heartdisease'),('cp','heartdisease'),('heartdiseas
e','restecg'),('heartdisease','chol')])
print('\nLearning CPD using Maximum likelihood estimators')
model.fit(heartDisease,estimator=MaximumLikelihoodEstimator)

print('\n Inferencing with Bayesian Network:')
HeartDiseasetest_infer = VariableElimination(model)

print('\n 1. Probability of HeartDisease given evidence= restecg')
q1=HeartDiseasetest_infer.query(variables=['heartdisease'],evidence={'restecg':1})
print(q1)

print('\n 2. Probability of HeartDisease given evidence= cp ')
q2=HeartDiseasetest_infer.query(variables=['heartdisease'],evidence={'cp':2})
print(q2)
```

Learning CPD using Maximum likelihood estimators

Inferencing with Bayesian Network:

1. Probability of HeartDisease given evidence= restecg

heartdisease	phi(heartdisease)
heartdisease(0)	0.1012
heartdisease(1)	0.0000
heartdisease(2)	0.2392
heartdisease(3)	0.2015
heartdisease(4)	0.4581

2. Probability of HeartDisease given evidence= cp

heartdisease	phi(heartdisease)
heartdisease(0)	0.3610
heartdisease(1)	0.2159
heartdisease(2)	0.1373
heartdisease(3)	0.1537
heartdisease(4)	0.1321

Program-10

Consider the "airline-passengers.csv "dataset. Write a program to implement LSTM (Short Long-term Memory) Network, the task is to predict the number of international airline passengers in units of 1,000. The data ranges from January 1949 to December 1960, or 12 years, with 144 observations

The problem you will look at in this post is the International Airline Passengers prediction problem.

This is a problem where, given a year and a month, the task is to predict the number of international airline passengers in units of 1,000. The data ranges from January 1949 to December 1960, or 12 years, with 144 observations.

- Download the dataset (save as "*airline-passengers.csv*").
  Below is a sample of the first few lines of the file.

```
1 "Month","Passengers"
2 "1949-01",112
3 "1949-02",118
4 "1949-03",132
5 "1949-04",129
6 "1949-05",121
```

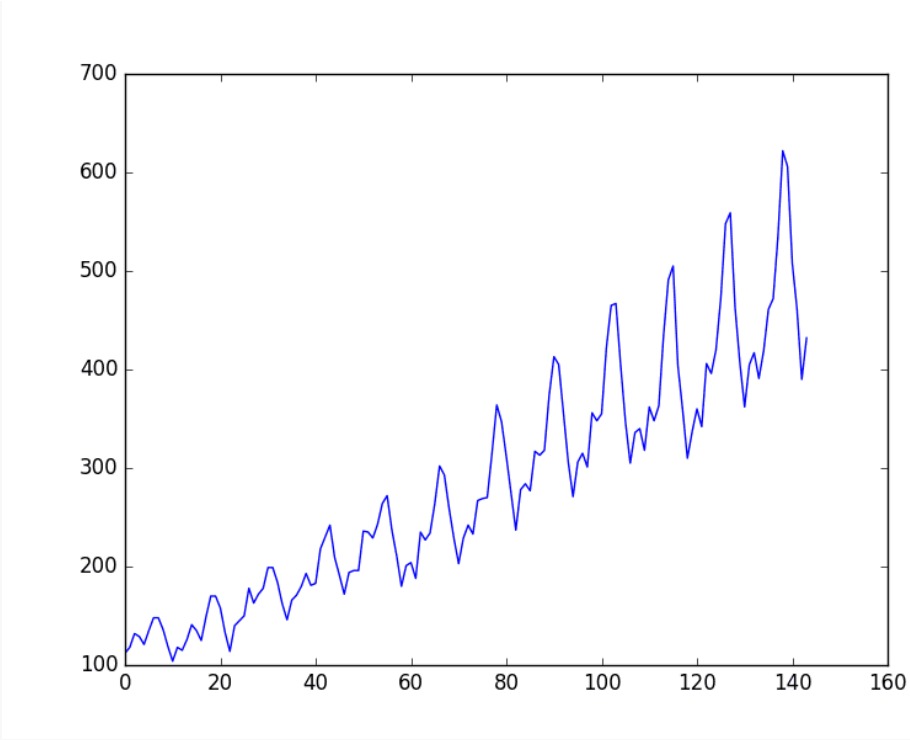
You can load this dataset easily using the Pandas library. You are not interested in the date, given that each observation is separated by the same interval of one month. Therefore, when you load the dataset, you can exclude the first column.

Once loaded, you can easily plot the whole dataset. The code to load and plot the dataset is listed below.

```
1 import pandas
2 import matplotlib.pyplotasplt
3 dataset=pandas.read_csv('airline-passengers.csv',usecols=[1],engine='python')
4 plt.plot(dataset)
5 plt.show()
```

You can see an upward trend in the dataset over time.

You can also see some periodicity in the dataset that probably corresponds to the Northern Hemisphere vacation period.



Plot of the airline passengers dataset

Let’s keep things simple and work with the data as-is.

Normally, it is a good idea to investigate various data preparation techniques to rescale the data and make it stationary

Program-11

Project Statement Project Title – Stock Market Prediction Stock price prediction is one among the complex machine learning problems. It depends on a large number of factors which contribute to changes in the supply CO3,4 and demand. This paper presents the technical analysis of the various strategies proposed in the past, for predicting the price of a stock, and evaluation of a novel approach for the same. Stock prices are represented as time series data and neural networks are trained to learn the patterns from trends. Along with the numerical analysis of the stock trend, this research also considers the textual analysis of it by analyzing the public sentiment from online news sources and blogs. Utilizing both this information, a merged hybrid model is built which can predict the stock trend more accurately

Stock Price Prediction :

Stock (also known as equity) is a security that represents the ownership of a fraction of a corporation. This entitles the owner of the stock to a proportion of the corporation's assets and profits equal to how much stock they own. Units of stock are called "shares." A stock is a general term used to describe the ownership certificates of any company. Stock prices change everyday by market forces. By this we mean that share prices change because of supply and demand. If more people want to buy a stock (demand) than sell it (supply), then the price moves up. Conversely, if more people wanted to sell a stock than buy it, there would be greater supply than demand, and the price would fall. Understanding supply and demand is easy. So, why do stock prices change? The best answer is that nobody really knows for sure. Some believe that it isn't possible to predict how stocks will change in price while others think that by drawing charts and looking at past price movements, you can determine when to buy and sell. The only thing we do know as a certainty is that stocks are volatile and can change in price extremely rapidly.

Understanding the Problem Statement

We'll dive into the implementation part of this Project soon, but first it's important to establish what we're aiming to solve. Broadly, stock market analysis is divided into two parts – Fundamental Analysis and Technical Analysis. Fundamental Analysis involves analyzing the company's future profitability on the basis of its current business environment and financial performance. Technical Analysis, on the other hand, includes reading the charts and using statistical figures to identify the trends in the stock market. As you might have guessed, our focus will be on the technical analysis and visualization part. We'll be using a dataset from Google stock Price test and train.

Implementation:

1.UsingSckiit Learning(Machine Learning model)

2.DataPreprocessing using dataset

3.Visualization of Dataset

4.Feature Scaling

5.Preparing the Datasets for training

6.Reshaping the datasets

7.Model development

8.Implementation of sequential, dense, LSTM and dropout.

9.Preprocessing the Data

10.Predicting the Output

11.Result visualization

Stock Price as a Time Series Data

Treating stock data as time-series, one can use past stock prices (and other parameters) to predict the stock prices for the next day or week. Machine learning models such as Recurrent Neural Networks (RNNs) or LSTMs are popular models applied to predicting time series data such as weather forecasting, election results, house prices, and, of course, stock prices. The idea is to weigh out the importance of recent and older data and determine which parameters affect the "current" or "next" day prices the most. The machine learning model assigns weights to each market feature and determines how much history the model should look at for stock market prediction using machine learning project to work out.

THANKS FOR READING

BBU